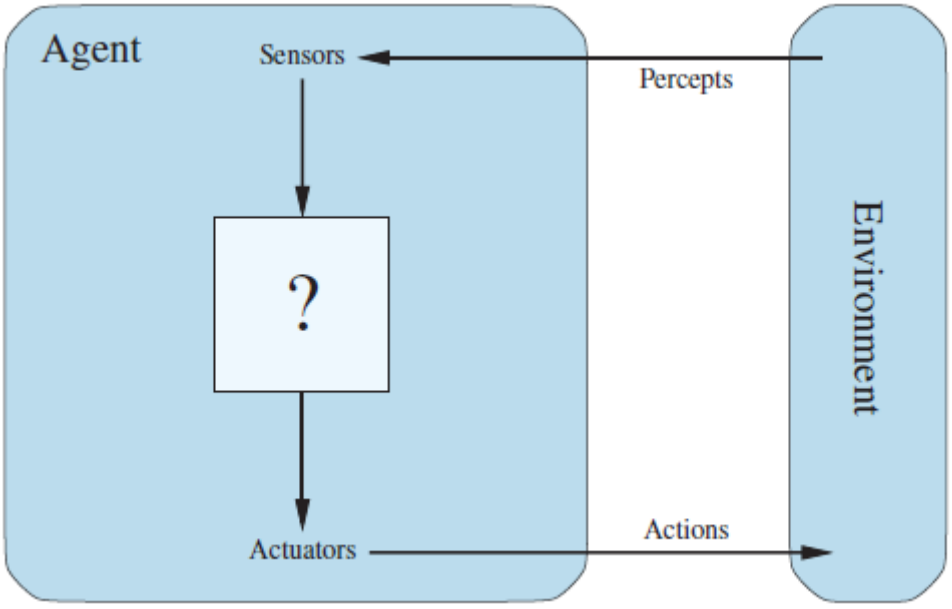
Chapter 2

2nd chapter : the nature of agents, perfect or otherwise, the diversity of environments, and the resulting menagerie of agent type
Agent: anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.

- **environment** : The environment could be everything—the entire universe! In practice it is just that part of the universe whose state we care about when designing this agent **that affects what the agent perceives and that is affected by the agent's actions**
- **percept** : what the agent is perceiving
- **percept sequence**: history of everything that the agent has perceived .
- The agent's actions depend of it's prior knowledge and on the entire precept sequence . "an agent's behavior is described by the *agent function* that maps any given **percept sequence to an action**."



Agent function : for every percept sequence it maps it to a specific function
we can implement it using an **agent program** ..

> **Note**
>
> **The agent function** is an abstract mathematical description; **the agent program** is a concrete implementation, running within some physical system

- What is the right way to fill out the table? In other words, what makes an agent good or bad, intelligent or stupid? We answer these questions in the next section.

Percept sequence	Action
$[A, Clean]$	*Right*
$[A, Dirty]$	*Suck*
$[B, Clean]$	*Left*
$[B, Dirty]$	*Suck*
$[A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Dirty]$	*Suck*
⋮	⋮
$[A, Clean], [A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Clean], [A, Dirty]$	*Suck*
⋮	⋮

Partial tabulation of a simple agent function for the vacuum-cleaner world shown

- The agent cleans the current square if it is dirty, otherwise it moves to the other square. Note that the table is of unbounded size unless there is a restriction on the length of possible percept sequences.

Good behavior The concept of rationality

A **rational agent** is one that does the *right thing.* Obviously, doing the right thing is better Rational agent
than doing the wrong thing, but *what does it mean to do the right thing?*

- we we can define the right thing using some measure like performance measure

performance measure :

- **consequentialism**: we evaluate an agent's behavior by its consequences.
  "so the notion of rationality as applied to humans has to do with their success in choosing actions that produce sequences of environment states that are desirable from their point of view."
- machines have no desire whatsoever , so when we speak about desirability we use the performance measure that is initially at least , in the mind of the designer of that machine .

> **Note**
>
> "*As a general rule, it is better to design performance measures according to what one actually wants to be achieved in the environment, rather than according to how one thinks the agent should behave.*"

- keep in mind that we are not successful each time when choosing the performance measure !

Rationality

- what's **rational** depends on :
  - the performance measure
  - agent's prior knowledge of the enviornment
  - Actions that the agent can perform
  - Agent's percept sequence to date

Rational Agent : For each possible **percept sequence**, a rational agent should select an action that is expected
to **maximize its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

omniscience , learning , autonomy

omniscience :

- **An omniscient agent** knows the actual outcome of its actions and can act accordingly.

> **Rationality and Perfection**
>
> Rationality maximizes expected performance, while perfection maximizes actual performance.

Information Gathering : doing actions in order to modify future percepts

- A second example of information gathering is provided by the exploration that must be undertaken by a vacuum-cleaning agent in an initially unknown environment.

Learning : The agent's initial configuration could reflect some
prior knowledge of the environment, but as the agent gains experience this may be modified and augmented.

Autonomy : A rational agent should be autonomous—it should learn what it can to *compensate* for partial or incorrect prior knowledge.

> "Thus, the agent's initial randomness is not irrational; it is a step toward becoming a fully rational and autonomous agent."

6. Example: A Reinforcement Learning Agent

- Consider a reinforcement learning agent in a new environment:
  - Initially, the agent has no knowledge of which actions are good or bad, so it explores randomly.
  - As it interacts with the environment, it receives rewards or penalties, which it uses to update its policy.
  - Over time, the agent learns to select actions that maximize its cumulative reward, becoming both rational and autonomous.

1. PEAS Framework

- To design a rational agent, we first define its task using the **PEAS** framework:
  - **Performance Measure**: How success is evaluated.
  - **Environment**: The external world the agent interacts with.
  - **Actuators**: The tools the agent uses to perform actions.
  - **Sensors**: The tools the agent uses to perceive the environment.
- *Example*: For a self-driving car:
  - **Performance Measure**: Safety, efficiency, comfort.
  - **Environment**: Roads, traffic, pedestrians.
  - **Actuators**: Steering wheel, brakes, accelerator.
  - **Sensors**: Cameras, radar, GPS.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Figure 2.4 PEAS description of the task environment for an automated taxi driver.

Properties of Environments

Environments can be categorized based on the following properties:

Fully Observable vs. Partially Observable:

- **Fully Observable**: The agent can see the entire state of the environment at all times.
- **Partially Observable**: The agent has limited or incomplete information about the environment.

Remark

If the agent has no sensors at all then the environment is unobservable

Multi Vs. Single Agent :

- **Single Agent** : Only one agent operates, and its actions are not influenced by other agents.
  - No other agents are present.
  - Performance depends solely on the agent's actions.
  - Can be deterministic (predictable) or stochastic (uncertain).
- ***Example***: A vacuum cleaner robot cleaning a room alone. Its performance is based on how well it cleans, with no other agents affecting its task.
- **Multi-Agent** : Multiple agents interact, and their actions influence each other.
  - Agents can be **competitive**, **cooperative**, or both.
  - Actions of one agent affect the environment and other agents.
  - Rational behavior may involve communication, negotiation, or randomization.
- ***Examples***:
  - **Chess**: A competitive two-agent environment where one player's gain is the other's loss.
  - **Taxi Driving**: A partially cooperative and competitive environment. Drivers cooperate to avoid accidents but compete for parking spots or passengers.

Deterministic Vs. Stochastic :

- **Deterministic**: The next state is **fully predictable** based on the current state and the agent's action. No uncertainty.
  - ***Example***: Classic vacuum world.
- **Nondeterministic**: The next state is **unpredictable** due to randomness, partial observability, or external factors.
  - ***Example***: Taxi driving, where traffic or mechanical failures introduce uncertainty.
- A **stochastic** environment explicitly models uncertainty using probabilities (e.g., "25% chance of rain").
- **Key Point**: All stochastic environments are nondeterministic, but not all nondeterministic environments are stochastic.

Episodic Vs. Sequential

- **Episodic**: The agent's experience is divided into independent episodes. Each episode involves a single percept-action cycle, and actions in one episode do not affect future episodes.
  - ***Example***: An agent inspecting parts on an assembly line. Each decision (e.g., classifying a part as defective) is based only on the current part and doesn't influence the next part.
- **Sequential**: The agent's current action can influence future decisions and outcomes. These environments involve long-term consequences and dependencies.
  - ***Example***: Chess (each move affects the game's future state) and taxi driving (decisions like lane changes impact future traffic interactions

Static vs. Dynamic :

- **Static**: The environment does not change while the agent is deliberating.
- **Dynamic**: The environment can change while the agent is thinking or acting.
  - **Semidynamic**: The environment itself does not change, but the **agent's performance score** does over time.
    - ***Example***: Chess with a clock. The board state remains the same while the agent thinks, but the time limit affects the agent's performance.

Discrete vs. Continuous:

- **Discrete**: The environment has a finite number of distinct states, actions, and percepts.
  - ***Example*** : *Chess* The board has a finite number of states, and moves (actions) are discrete and well-defined.
- **Continuous**: The environment has an infinite number of possible states or actions (e.g., a self-driving car navigating a road).

Known Vs. Unknown :

- **Known Environment**: The agent has full knowledge of the environment's rules and outcomes (or probabilities) for all actions.
  - ***Example***: A chess agent knows all rules and move effects.
  - Even in **partially observable** known environments (e.g., solitaire), the agent knows the rules but may not see the full state (e.g., face-down cards).
- **Unknown Environment**: The agent doesn't know the environment's rules and must learn them through interaction.
  - ***Example***: A new video game where the agent sees the full state but doesn't know what actions do until tried.
  - The **performance measure** may also be unknown (e.g., a taxi driver learning passenger preferences or a virtual assistant adapting to user preferences).

→ parctice on examples for enviorment !

> **REMARK**
>
> The hardest environment for an agent is one that is **partially observable, multiagent, nondeterministic, sequential, dynamic, continuous, and unknown**.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

1. **Importance of Environment Properties**:
   - The properties of the environment shape the design of the agent and the algorithms it uses.
   - Example:
     - In *simple environments* (fully observable, deterministic, static), basic algorithms like search work well.
     - In *complex environments* (partially observable, stochastic, dynamic), advanced techniques like probabilistic reasoning or reinforcement learning are needed.
2. **Real-World Environments**:
   - Most real-world environments are *Complex*: **partially observable, stochastic, sequential, dynamic, continuous, and multi-agent.**
   - This complexity makes designing rational agents challenging, as they must handle uncertainty, adapt to changes, and interact with other agents.

Types of Agents

1. Simple Reflex Agents

- **How they work**: These agents make decisions based solely on the **current percept** (input) using predefined **condition-action rules** (if-then rules). They ignore percept history or future consequences.
- **Example**:
  - A **thermostat** turns on the heater if the temperature is below a certain level (condition: if cold → action: turn on heater).
  - A **vacuum cleaner** sucks dirt if the current location is dirty (condition: if dirty → action: suck).
- **Limitation**: They fail in partially observable environments because they don't consider past or future states.

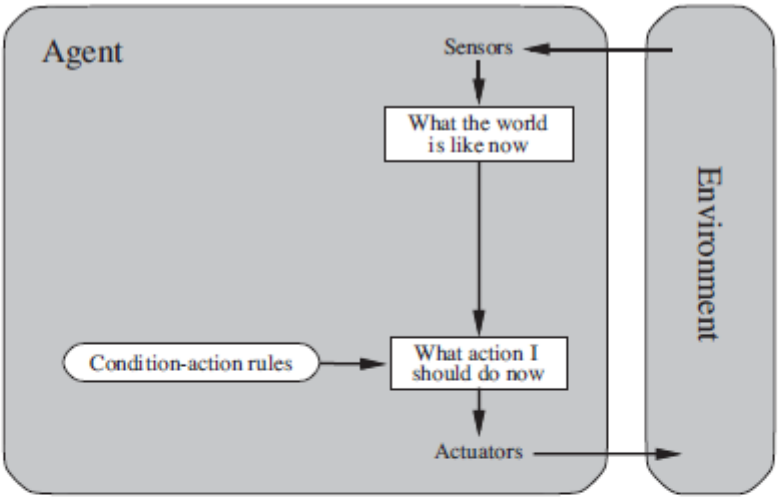


Figure 2.9 Schematic diagram of a simple reflex agent.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

---

2. Model-Based Reflex Agents

- **How they work**: These agents maintain an **internal state** to track aspects of the world they can't directly observe. They use a **model of the world** to update their internal state based on percept history and choose actions accordingly.
- **Example**:
  - A **self-driving car** uses past camera frames to detect if the car in front is braking (e.g., by comparing brake light patterns over time).
  - A **vacuum cleaner** remembers which rooms it has already cleaned to avoid repeating them.
- **Advantage**: Handles partially observable environments better than simple reflex agents.

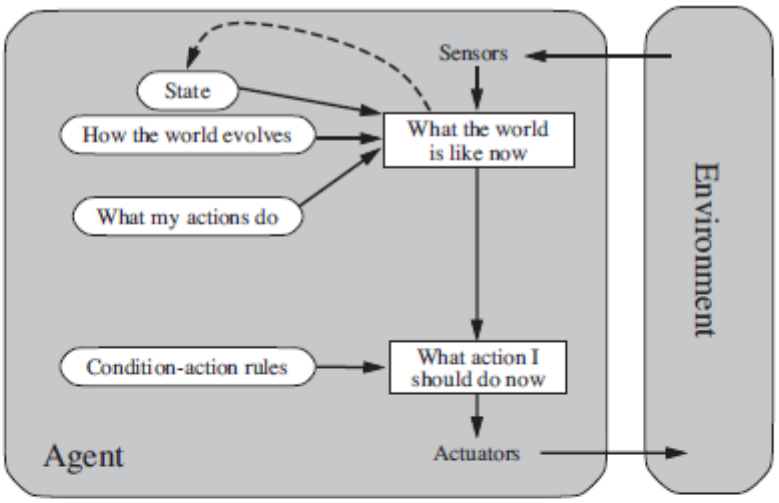


Figure 2.11 A model-based reflex agent.

function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
persistent: *state*, the agent's current conception of the world state
model, a description of how the next state depends on current state and action
rules, a set of condition–action rules
action, the most recent action, initially none

state ← UPDATE-STATE(*state*, *action*, *percept*, *model*)
rule ← RULE-MATCH(*state*, *rules*)
action ← *rule*.ACTION
return *action*

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

---

3. Goal-Based Agents

- **How they work**: These agents make decisions based on **goals** they need to achieve. They use a model of the world to plan actions that lead to the desired goal state.
- **Example**:
  - A **delivery robot** plans the shortest route to deliver a package to a specific location (goal: reach destination → action: follow optimal path).
  - A **chess-playing AI** evaluates moves to achieve the goal of checkmating the opponent.
- **Advantage**: Can handle complex tasks requiring planning and foresight.

Utility-Based Agents

- **How they work**: These agents make decisions by maximizing **utility**, a measure of how "happy" or satisfied the agent is in a given state. Unlike goal-based agents, which only care about achieving a goal, utility-based agents evaluate and compare different states based on their **quality** (e.g., speed, safety, cost). They choose actions that maximize **expected utility**, considering the probabilities and outcomes of each action.

- **Example**:

```
- A **self-driving taxi** doesn’t just aim to reach the destination (goal). It also considers factors like safety, fuel efficiency, and travel time. For instance, it might choose a longer route to avoid traffic or reduce risk, even if it takes more time, because the overall utility (safety + efficiency) is higher.

- A **trading bot** doesn’t just aim to make a profit (goal). It evaluates trades based on risk, potential reward, and market conditions to maximize overall utility (profit minus risk).
```

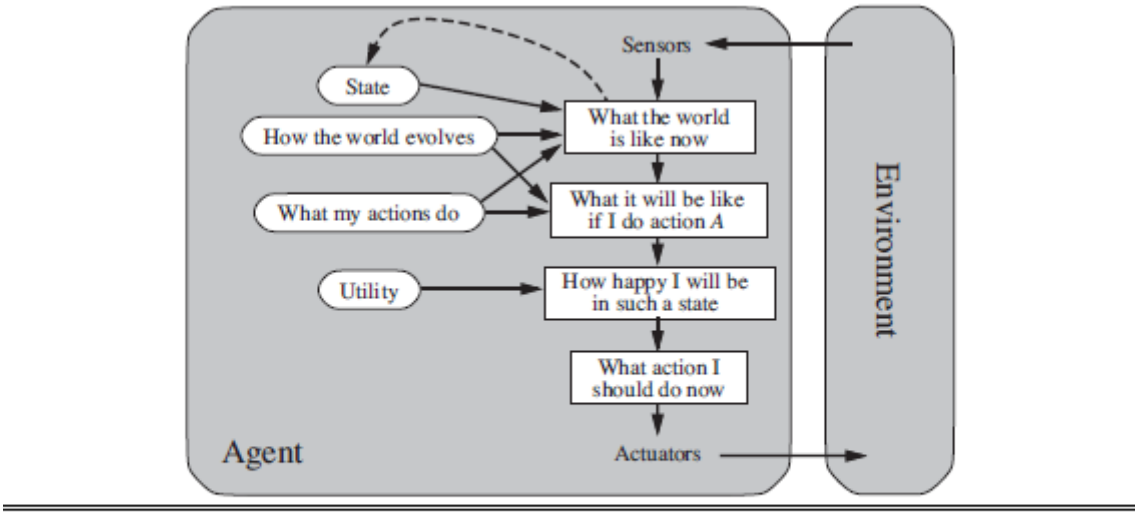


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Learning Agents

- **How they work**: Learning agents improve their performance over time by learning from experience. They consist of four key components:
  1. **Learning Element**: Responsible for improving the agent’s knowledge or behavior.
  2. **Performance Element**: Makes decisions and takes actions based on current knowledge (like a regular agent).
  3. **Critic**: Provides feedback on how well the agent is performing (e.g., success or failure).
  4. **Problem Generator**: Suggests new actions or experiences to help the agent learn.
- **Example**:
  - A **recommendation system** (like Netflix or Spotify) learns user preferences over time. It starts with basic recommendations, receives feedback (e.g., what users watch or skip), and improves its suggestions.
  - A **self-driving car** learns from driving experiences, such as how to handle specific road conditions or avoid obstacles, improving its driving skills over time.

Summary Table:

Agent Type	Key Feature	Example
Simple Reflex Agent	Acts based on current percept only.	Thermostat, vacuum cleaner.
Model-Based Agent	Maintains internal state for partial observability.	Self-driving car, advanced vacuum.
Goal-Based Agent	Plans actions to achieve specific goals.	Delivery robot, chess AI.
Utility-Based Agent	Maximizes utility (quality of outcomes).	Self-driving taxi, trading bot.
Learning Agent	Improves over time through experience.	Recommendation system, self-driving car.

1. Atomic Representation

- **What it is**: The world is represented as indivisible states with no internal structure.
- **Example**: Planning a driving route by only considering city names (e.g., "City A" to "City B").
- **Use Cases**: Search algorithms, game-playing, Hidden Markov Models, Markov Decision Processes.

---

2. Factored Representation

- **What it is**: States are split into variables/attributes with values (e.g., gas level, GPS coordinates).
- **Example**: Representing a car's state with attributes like fuel level, oil warning light status, and location.
- **Advantages**: More expressive than atomic; can handle uncertainty.
- **Use Cases**: Constraint satisfaction, propositional logic, planning, Bayesian networks.

---

3. Structured Representation

- **What it is**: Represents objects and their relationships (e.g., cows, trucks, and how they interact).
- **Example**: Describing a complex scenario like "Truck blocking a driveway due to a loose cow."
- **Advantages**: Highly expressive; captures real-world complexity concisely.
- **Use Cases**: Relational databases, first-order logic, natural language understanding, knowledge-based learning.

1. **Increasing Complexity**: Atomic → Factored → Structured.

2. **Increasing Expressiveness**: Structured representations can capture more detail concisely.
3. **Trade-off**: More expressive representations make reasoning and learning more complex.
4. **Combination**: Sometimes, multiple representations are used together for better results.

Example:

- **Atomic**: "City A" → "City B."
- **Factored**: "Car in City A with 50% fuel and working GPS."
- **Structured**: "Truck in City A blocking a driveway because of a loose cow."